

Material : To create web application

Topic : **Streamlit Framework**



Daniel
danielgenai77@gmail.com

Streamlit Framework

1. Framework.....	2
2. Streamlit framework.....	2
3. Simple Web Apps for ML Engineers.....	2
4. Why streamlit?	3
5. Steamlit environment setup	3
6. Hello World example.....	4
7. When happens, when we run Steamlit program?	5
7.1. Streamlit Starts Running	5
7.2. Re-running on File Change	5
7. 3. Shutting Down.....	5
8. Adding title to streamlit app.....	6
9. Add a second header and Subsection Heading	7
10. Creating variables and displaying	8
11. Widget	10
11.1. Text & Number input Widgets	11
11.2. Selection Widgets	13
11.3. Sliders widgets	15
11.4. Buttons and Triggers widgets	17
11.5. Date and Time Pickers widgets	19
11.6. File upload widgets	20
12. Streamlit Message Boxes: Success, Info, Warning, Error, Exception	23
13. Use cases.....	25
13.1. Shows the current time and updates every second.	25
13.2. Simple counter	26
13.3. Progress bar	27
13.4. Real-time Data Dashboard (e.g., Stock Prices or Crypto)	28

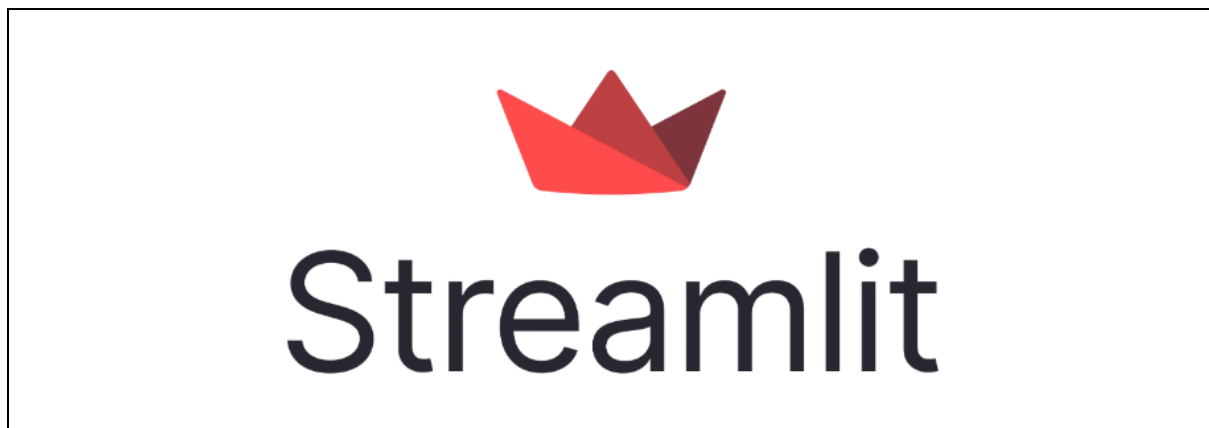
Streamlit Framework

1. Framework

- ✓ A framework is a reusable, structured platform.
- ✓ It provides a skeleton to develop software applications.
- ✓ So, developers can focus on the specific logic of their application.

2. Streamlit framework

- ✓ Streamlit is a Python-based framework.
- ✓ **Streamlit** is a free and open-source framework.
- ✓ By using this we can create interactive **web apps** for data science and machine learning without needing web development skills
- ✓ Logo



3. Simple Web Apps for ML Engineers

- ✓ It is a Python library made for machine learning engineers and data scientists.
- ✓ We can create interactive, great-looking apps with just a few lines of code.
- ✓ Perfect for displaying data and collecting input for models.

4. Why streamlit?

- ✓ Here are key reasons why Streamlit stands out for data science and ML deployment:
 - **No Frontend Skills Needed:** No HTML, CSS, or JavaScript required.
 - **Fast Prototyping:** Build interactive apps in minutes.
 - **Pythonic:** Write everything in pure Python.
 - **Data Science Friendly:** Seamlessly integrates with Pandas, Plotly, TensorFlow, and more.
 - **Easy Deployment:** Run locally or share online via Streamlit Cloud.
 - **Interactive Widgets:** Add sliders, buttons, and dropdowns with one line of code.
- ✓ So, Streamlit is perfect for quickly turning our ML models and data analyses into usable, shareable apps.

5. Streamlit environment setup

- ✓ We can install streamlit by following below command.

```
pip install streamlit
```

6. Hello World example

- ✓ We can write Hello World example by using `write()` method to **DeltaGenerator** class
- ✓ Under the hood, this imports a module, which internally sets up a special object (an instance of the DeltaGenerator class) that handles **UI** rendering.

Program Name	Streamlit Hello World example demo1.py
	<pre>import streamlit as st st.write("Hello World")</pre>
Run	<code>streamlit run demo1.py</code>
Output	Hello World

7. When happens, when we run Steamlit program?

- ✓ We can run streamlit program by following below command.

```
streamlit run filename.py
```

7.1. Streamlit Starts Running

- ✓ Streamlit launches a local web server on port 8501 (by default).
- ✓ We can open **http://localhost:8501** in your browser to interact with your app.

```
C:\Users\Daniel\Desktop\CODING>streamlit run demo.py

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://192.168.1.10:8501
```

7.2. Re-running on File Change

- ✓ If we done any changes in .py file, Streamlit will detect them and auto-re-run the app.

7. 3. Shutting Down

- ✓ To stop the app, press **Ctrl + C** in the CMD.

8. Adding title to streamlit app

- ✓ `st.title()` is predefined method in **DeltaGenerator** class.
- ✓ `st.title()` adds a large header at the top of the page.
- ✓ It's like an **<h1>** in HTML.

Program Name Streamlit Hello World & title example
demo2.py

```
import streamlit as st
```

```
st.title("Welcome to Streamlit")  
st.write("Hello World")
```

Run streamlit run demo1.py

Output

Welcome to Streamlit

Hello World

9. Add a second header and Subsection Heading

- ✓ We can display a section-level heading in Streamlit app.
- ✓ `st.header()` is predefined method in **DeltaGenerator** class.
 - By using this we can display a section-level heading in Streamlit app, slightly smaller than the main title.
 - It's like an `<h2>` in HTML.
- ✓ `st.subheader()` is predefined method in **DeltaGenerator** class.
 - By using this we can display a subsection heading in Streamlit app.
 - It's like an `<h3>` in HTML.

Program Name Streamlit title, heading & subheading demo3.py

```
import streamlit as st

st.title("This is Title")
st.header("This is Header")
st.subheader("This is Sub-Header")
```

Run streamlit run demo1.py

Output

This is Title

This is Header

This is Sub-Header

10. Creating variables and displaying

- ✓ We can create variables and display those values over app

Program Name Creating variable and displaying demo4.py

```
import streamlit as st
```

```
a = 10
```

```
b = 1.23
```

```
c = "Daniel"
```

```
st.write(a)
```

```
st.write(b)
```

```
st.write(c)
```

Run streamlit run demo1.py

Output

```
10
```

```
1.23
```

```
Daniel
```

Program Name Creating variable and displaying demo5.py

```
import streamlit as st
```

```
a = 10
```

```
b = 1.23
```

```
c = "Daniel"
```

```
st.write(a, b, c)
```

Run streamlit run demo1.py

Output

```
10 1.23 Daniel
```

11. Widget

- ✓ A widget is a small interactive user interface (UI) element that lets users' input, select, or control something in an app.
- ✓ In the context of Streamlit:
 - A widget is a built-in functionality (`st.button()` or `st.text_input()`) that:
 - Takes input from the user (e.g., typing, clicking, selecting)
 - Returns that input as a value in your code
 - Triggers your app to react or update

Why are Widgets Useful?

- ✓ Make your app interactive
- ✓ Capture user input
- ✓ Dynamically change output based on user actions

Common and Important Streamlit Widgets

1. Text & Number Input
2. Selection Widgets
3. Sliders
4. Buttons and Triggers
5. Date and Time Pickers
6. File Upload

11.1. Text & Number input Widgets

- ✓ These widgets allow users to collect user input directly in your web app, such as names, numbers, or feedback.
 - `st.text_input()`:
 - Accepts single-line text input as name, email, comments
 - `st.number_input()`
 - Accepts integer or float values as age, salary, score
 - `st.text_area()`
 - Accepts multi-line text input as feedback, notes

Program Name Text & Number input widgets
demo6.py

```
import streamlit as st

name = st.text_input("Enter your name:")
st.write("Name:", name)

age = st.number_input("Enter your age:", min_value = 0,
max_value=120, step = 1)
st.write("Age:", age)

feedback = st.text_area("Your feedback:")
st.write("Feedback received:", feedback)
```

Run streamlit run demo1.py

Output

Enter your name:

Daniel

Name: Daniel

Enter your age:

100

Age: 100

Your feedback:

Its really good

Feedback received: Its really good

11.2. Selection Widgets

- ✓ These widgets allow users to select options from predefined lists or toggle choices in your app.
 - `st.selectbox()`
 - Choose one item from a list like select country, category
 - `st.multiselect()`
 - Choose multiple items like pick skills, favourite colors
 - `st.radio()`
 - Single choice with radio buttons like gender, Yes/No options
 - `st.checkbox()`
 - On/Off toggle. Accept terms, Show chart

Program Name

Selection widgets
demo7.py

```
import streamlit as st
```

```
a = ["USA", "Canada", "UK", "India"]  
b = ["Python", "Java", "C++", "JavaScript"]  
c = ["Male", "Female"]
```

```
country = st.selectbox("Select your country:", a)  
st.write("Country:", country)
```

```
skills = st.multiselect("Pick your skills:", b)  
st.write("Skills:", skills)
```

```
gender = st.radio("Select your gender:", c)  
st.write("Gender:", gender)
```

```
terms = st.checkbox("Accept Terms and Conditions")  
st.write("Terms accepted:", terms)
```

Run

```
streamlit run demo1.py
```

Output

Select your country:

Canada

Country: Canada

Pick your skills:

Python ×

JavaScript ×

Skills:

▶ [. . .]

Select your gender:

☒ Male

☐ Female

Gender: Male

☒ Accept Terms and Conditions

Terms accepted: True

11.3. Sliders widgets

- ✓ These widgets allow the users select a value or range by sliding a handle along a bar, making it easy to pick numbers or dates within limits.
 - `st.slider()`:
 - Choose a numeric or date range. Age range, date range
 - `st.select_slider()`
 - Choose from custom steps. Rating: Bad → Good → Excellent

Program Name

Sliders widgets
demo8.py

```
import streamlit as st
```

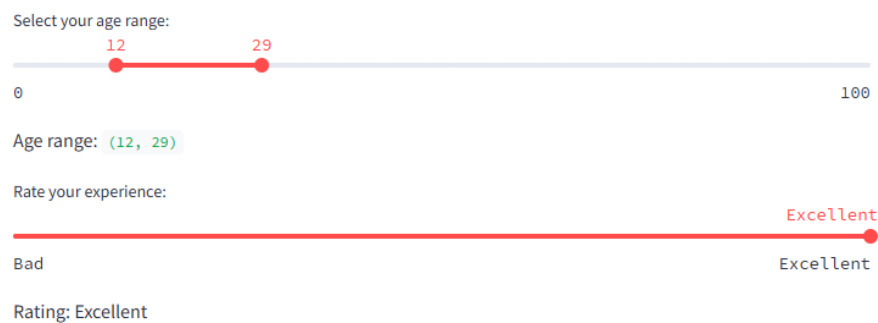
```
age_range = st.slider("Select your age range:", 0, 100, (25, 50))  
st.write("Age range:", age_range)
```

```
rating = st.select_slider(  
    "Rate your experience:",  
    options = ["Bad", "Average", "Good", "Excellent"]  
)  
st.write("Rating:", rating)
```

Run

streamlit run demo1.py

Output



Select your age range:

0 100

Age range: (12, 29)

Rate your experience:

Bad Excellent

Rating: Excellent

11.4. Buttons and Triggers widgets

- ✓ These widgets are used to trigger actions like submitting a form or running a function when the user clicks a button.
 - `st.button()`:
 - Triggers an action immediately. Submit form, Start analysis
 - `st.form() + st.form_submit_button()`:
 - Groups input fields and submits them together. Form submission, Multi-field input

Program Name Button & Trigger widgets
demo9.py

```
import streamlit as st

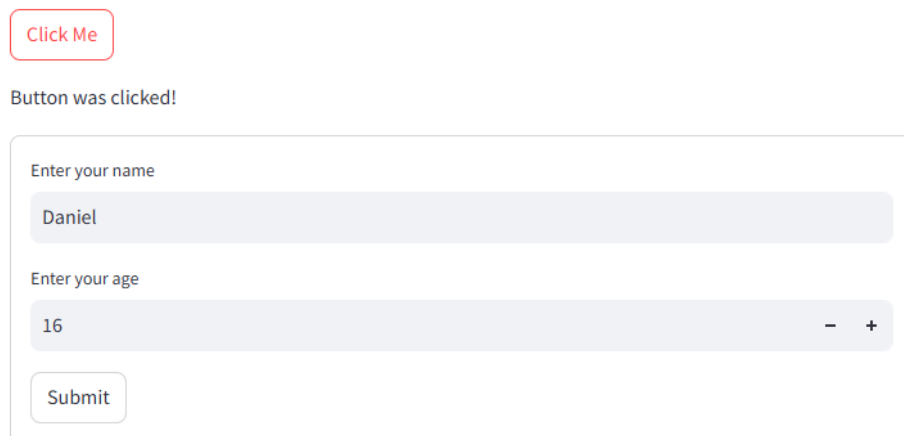
if st.button("Click Me"):
    st.write("Button was clicked!")

with st.form("user_form"):
    name = st.text_input("Enter your name")
    age = st.number_input("Enter your age", min_value = 0)
    submitted = st.form_submit_button("Submit")

if submitted:
    st.write(f"Name: {name}, Age: {age}")
```

Run streamlit run demo1.py

Output



Click Me

Button was clicked!

Enter your name

Daniel

Enter your age

16 - +

Submit

11.5. Date and Time Pickers widgets

- ✓ These widgets let users select specific dates and times.
- ✓ Useful for scheduling, time-based input.
 - `st.date_input()`:
 - Select a date. Pick DOB
 - `st.time_input()`:
 - Select a time. Pick a meeting time

Program Name Date and Time pickers
demo10.py

```
import streamlit as st
import datetime as dt
```

```
a = dt.date(2000, 1, 1)
dob = st.date_input("Select your date of birth:", a)
st.write("Date of Birth:", dob)
```

```
meeting_time = st.time_input("Select meeting time:")
st.write("Meeting Time:", meeting_time)
```

Run `streamlit run demo1.py`

Output

Select your date of birth:

2000/01/14

Date of Birth: 2000-01-14

Select meeting time:

00:15

Meeting Time: 00:15:00

11.6. File upload widgets

- ✓ Allows users to upload files (e.g., CSV, images, PDFs) directly into the app for processing or display.
 - `st.file_uploader()`: Upload a file.
 - CSV, image, Excel, PDF

Program Name Loading csv file
demo11.py

```
import streamlit as st
import pandas as pd

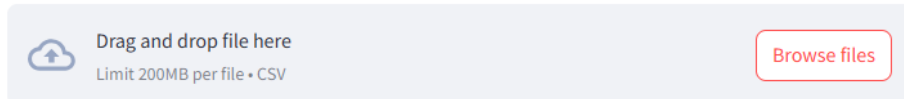
uploaded_file = st.file_uploader("Upload a CSV file", type =
["csv"])

if uploaded_file is not None:
    df = pd.read_csv(uploaded_file)
    st.write("Uploaded Data:")
    st.dataframe(df)
```

Run streamlit run demo1.py

Output

Upload a CSV file



 sales1.csv 20.2KB ×

Uploaded Data:

	Order ID	Customer Name	Product	Quantity
0	166837	Veeru	34in Ultrawide Monitor	2
1	166838	Tarun	Samsung m10	3
2	166839	Kedar	20in Monitor	1
3	166840	Lavanya	iPhone 11	3
4	166841	Venu	Macbook Pro Laptop	2
5	166842	Venu	Bose SoundSport Headphones	2

Program Name

Loading image file
demo12.py

```
import streamlit as st
from PIL import Image
```

```
uploaded_image = st.file_uploader("Upload an image",
type=["jpg", "jpeg", "png"])
```

```
if uploaded_image is not None:
    image = Image.open(uploaded_image)
    st.image(image, caption = "Uploaded Image",
    use_container_width = True)
```

Run

streamlit run demo1.py

Output

Upload an image



Drag and drop file here

Limit 200MB per file • JPG, JPEG, PNG

Browse files



JESUS IMAGES.PNG 0.6MB



12. Streamlit Message Boxes: Success, Info, Warning, Error, Exception

- ✓ Streamlit provides built-in methods to display **message** boxes with different visual styles to communicate **status**, alerts, or logs directly in the app interface.
 - `st.success()`: Show a success message (e.g. "Saved!"). **Green**
 - `st.info()`: Show general information or tips. **Blue**
 - `st.warning()`: Show a warning or caution message. **Yellow**
 - `st.error()`: Show an error or failure message. **Red**
 - `st.exception()`: Display Python exception details. **Red/Debug**

Program Streamlit message boxes
Name demo12.py

```
import streamlit as st
```

```
st.success("Operation completed successfully.")  
st.info("This app shows sales trends.")  
st.warning("Make sure the file is uploaded.")  
st.error("Something went wrong!")
```

```
try:  
    print(a)  
except NameError as ne:  
    st.exception(ne)
```

Run streamlit run demo1.py

Output

Operation completed successfully.

This app shows sales trends.

Make sure the file is uploaded.

Something went wrong!

NameError: name 'a' is not defined

Traceback:

```
File "C:\Users\Daniel\Desktop\CODING\demo.py", line 9, in <module>
    print(a)
    ^
```

13. Use cases

13.1. Shows the current time and updates every second.

Program Name Shows the current time and updates every second
demo11.py

```
import streamlit as st
import time
from datetime import datetime
```

```
st.title("Real-Time Clock")
```

```
now = datetime.now().strftime("%H:%M:%S")
st.markdown(f"### {now}")
time.sleep(1)
st.rerun()
```

Run streamlit run demo1.py

Output

Real-Time Clock

11:54:40

13.2. Simple counter

Program Name Simple counter
demo11.py

```
import streamlit as st

st.title("Simple Counter")

if "count" not in st.session_state:
    st.session_state.count = 0

if st.button("Increment"):
    st.session_state.count += 1

st.write(f"Count: {st.session_state.count}")
```

Run streamlit run demo1.py

Output

Simple Counter ↗

Increment

Count: 12

13.3. Progress bar

Program Name Simple counter
demo11.py

```
import streamlit as st
import time

st.title("Progress Bar")

progress = st.progress(0)

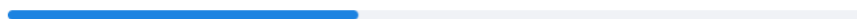
for i in range(100):
    progress.progress(i + 1)
    time.sleep(0.05)

st.write("Finished")
```

Run streamlit run demo1.py

Output

Progress Bar



13.4. Real-time Data Dashboard (e.g., Stock Prices or Crypto)

- ✓ Pulls live data from an API (e.g., Alpha Vantage, Binance) and updates prices, charts, and trends in real-time.

Key Features

- ✓ Auto-refresh using time-based loops
- ✓ Line chart updates with `st.line_chart()` or `st.plotly_chart()`
- ✓ Uses APIs like `yfinance`, `cryptocompare`, etc.
- ✓ <https://finance.yahoo.com/quote/AAPL/>

Install below package

```
pip install yfinance
```

Program Name	Real-time Data Dashboard e.g., Stock Prices demo11.py <pre>import streamlit as st import yfinance as yf import time st.title("Live Stock Price") symbols = ["AAPL", "GOOGL", "MSFT", "AMZN", "TSLA", "META", "NVDA"] symbol = st.selectbox("Select stock symbol", options = symbols) placeholder = st.empty() if symbol: while True: data = yf.download(tickers = symbol, period = "1d", interval = "1m") placeholder.line_chart(data["Close"]) time.sleep(60)</pre>
Run	streamlit run demo1.py
Output	

Live Stock Price

Select stock symbol

AAPL

